

DAY 2 PYTHON

1. LIST

A **list** is a collection of items stored in a single variable.

```
>>> files = ["customers.csv", "orders.csv", "products.csv"]
>>>
>>> print(files)
['customers.csv', 'orders.csv', 'products.csv']
>>> █
```

2. List Indexing

Indexing is used to access individual elements in a list. Python assigns a position number (index) to each element, starting from **0**.

```
>>> files = ["customers.csv", "orders.csv", "products.csv"]
>>>
>>> print(files[0])
customers.csv
>>>
>>> print(files[1])
orders.csv
>>> print(files[2])
products.csv
>>> █
```

3. Negative Index

Negative indexing allows you to access list elements from the **end of the list** instead of the beginning.

```
>>> files = ["customers.csv", "orders.csv", "products.csv"]
>>>
>>> print (files[-1])
products.csv
>>>
>>> print (files[-2])
orders.csv
>>>
>>> print (files[-3])
customers.csv
>>> █
```

4. len()

Syntax - len(object)

The **len()** function is used to find the **number of items (length)** in a list, string, tuple, dictionary, or other collection.

```
>>> files = ["customers.csv", "orders.csv", "products.csv"]
>>>
>>> print(len(files))
3
```

5. append()

Syntax - list_name.append(item)

The **append()** method is used to **add a new element to the end of a list**.

```
>>> files = ["customers.csv", "orders.csv"]
>>>
>>> files.append("prodcuts.csv")
>>>
>>> print(files)
['customers.csv', 'orders.csv', 'prodcuts.csv']
>>>
```

6. remove()

The **remove()** method is used to **remove a specific element by its value** from a list.

Syntax - list_name.remove(value)

```
>>> files = ["customers.csv", "orders.csv", "products.csv"]
>>>
>>> files.remove("orders.csv")
>>>
>>> print(files)
['customers.csv', 'products.csv']
```

7. Change a Value in a List

Python lists are **mutable**, which means you can change their elements after the list is created.

Syntax - list_name[index] = new_value

```
>>> files = ["customers.csv", "orders.csv", "products.csv"]
>>>
>>> files[1] = "sales.csv"
>>>
>>> print(files)
['customers.csv', 'sales.csv', 'products.csv']
>>>
>>> files[0] = "client.csv"
>>>
>>> print(files)
['client.csv', 'sales.csv', 'products.csv']
```

8. List Slicing

List slicing allows you to extract a portion (sublist) of a list using a range of indexes.

Syntax - list[start:stop]

```
>>> files = ["customers.csv", "orders.csv", "products.csv", "sales.csv", "employees.csv"]
>>>
>>> print(files[1:4])
['orders.csv', 'products.csv', 'sales.csv']
>>> print(files[2:3])
['products.csv']
>>> print(files[4:1])
[]
>>> print(files[1:3])
['orders.csv', 'products.csv']
```

9. Loop Through a List

A **loop** is used to go through each element in a list one by one. The most common way is using a **for loop**.

Syntax:

```
for item in list_name:
    print(item)
```

```

>>> files = ["customers.csv", "orders.csv", "products.csv", "sales.csv", "employees.csv"]
>>>
>>> for file in files:
...     print(file)
...
customers.csv
orders.csv
products.csv
sales.csv
employees.csv

```

10.AWS Data Engineering Example with List

In AWS Data Engineering, lists are commonly used to store file names, S3 paths, table names, or data sources that need to be processed.

Processing Files from Amazon S3 : daily data files are stored in an S3 bucket:

```

>>> s3_files = ["customers.csv", "orders.csv", "products.csv", "sales.csv", "employees.csv"]
>>>
>>> print(s3_files)
['customers.csv', 'orders.csv', 'products.csv', 'sales.csv', 'employees.csv']

```

Loop Through Files: A Data Engineer may need to process each file one by one.

```

>>> s3_files = ["customers.csv", "orders.csv", "products.csv", "sales.csv", "employees.csv"]
>>>
>>> for file in s3_files:
...     print("processing_files:", file)
...
processing_files: customers.csv
processing_files: orders.csv
processing_files: products.csv
processing_files: sales.csv
processing_files: employees.csv

```

Add a New Incoming File: When a new file arrives in S3:

```
>>> s3_files = ["customers.csv", "orders.csv", "products.csv", "sales.csv", "employees.csv"]
>>>
>>> s3_files.append("stored_data.csv")
>>>
>>> print(s3_files)
['customers.csv', 'orders.csv', 'products.csv', 'sales.csv', 'employees.csv', 'stored_data.csv']
>>>
```

Count Total Files

```
>>> print("Total Files:", len(s3_files))
Total Files: 6
>>>
```

11.Tuple

Tuples are used to store multiple items in a single variable. A tuple is a collection which is ordered and **unchangeable**.

While lists use square brackets [], tuples use parentheses ().

```
>>> thistuple = ("total_orders", "data", "sales", "products")
>>>
>>> print(thistuple)
['total_orders', 'data', 'sales', 'products']
>>>
```

12.Set

A **set** is a collection of **unique items**. Sets are **unordered**, **mutable**, and **do not allow duplicate values**.

```
>>> total_files = ["customers.csv", "orders.csv", "products.csv", "sales.csv", "orders.csv", "sales.csv", "employees.csv"]
>>>
>>> unique_files = set(total_files)
>>>
>>> print(unique_files)
{'products.csv', 'employees.csv', 'orders.csv', 'customers.csv', 'sales.csv'}
>>>
```

13)Dictionary

Dictionaries are used to store data values in key:value pairs.

- Keys must be unique.
- Values can be of any data type.
- Dictionaries are mutable (can be changed).

Dictionaries use curly braces {}.

```
>>> files_info = {  
...     "File_name": "Employee.csv",  
...     "Emp_id": "101",  
...     "Emp_name": "John",  
...     "city": "England"  
... }  
>>> print(files_info)  
{'File_name': 'Employee.csv', 'Emp_id': '101', 'Emp_name': 'John', 'city': 'England'}
```

14) Access Dictionary Values

A dictionary stores data as **key-value pairs**. To retrieve a value, you use its **key**.

```
>>> file_info = {  
...     "File_name": "Employee.csv",  
...     "Emp_id": "101",  
...     "Emp_name": "John",  
...     "City": "England"  
... }  
>>>  
>>>  
>>> print(file_info["File_name"])  
Employee.csv  
>>>  
>>> print(file_info["Emp_name"])  
John  
>>>
```

15) Add a New Key and Value

you can add a new key-value pair to a dictionary by assigning a value to a new key.

Syntax: `dictionary_name["new_key"] = value`

- ❑ If the key does not exist, Python adds it.
- ❑ If the key already exists, Python updates its value.

```
>>> file_info = {
...     "File_name": "Employee.csv",
...     "Emp_id": "101",
...     "City": "England"
... }
>>>
>>> file_info["Emp_name"] = "John"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'set' object does not support item assignment
>>> file_info["Emp_name"] = "John"
>>>
>>> print(file_info)
{'File_name': 'Employee.csv', 'Emp_id': '101', 'City': 'England', 'Emp_name': 'John'}
```

16) Change Dictionary Value

Dictionaries allow us to modify the value of an existing key. This is done by directly assigning a new value to the key.

Syntax

`dictionary_name["key"] = new_value`

```
[ec2-user@ip-172-31-27-247 ~]$ python3
Python 3.9.25 (main, Jul 15 2026, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
>>>
>>>
>>> student = {
...     "Emp_ID": "101",
...     "Emp_Name": "John",
...     "Emp_Age": "20",
...     "City": "Ooty"
... }
>>>
>>> student["Emp_Age"] = "25"
>>>
>>> print(student)
{'Emp_ID': '101', 'Emp_Name': 'John', 'Emp_Age': '25', 'City': 'Ooty'}
>>>
>>>
```

17.AWS S3 Example Using Dictionary

A dictionary is useful for storing information about an S3 bucket, file, or object.

Why Dictionaries Are Important in AWS

Most AWS services return data in JSON format, which Python converts into dictionaries.

Examples:

- **S3 Object Details**
- **EC2 Instance Information**
- **Lambda Event Data**
- **DynamoDB Records**
- **API Responses**

how a Dictionary can be used to store information about an AWS S3 file.

```

>>> s3_file = {
...     "bucket_name": "company-raw-data",
...     "file_name": "customers.csv",
...     "record_count": 1000,
...     "status": "SUCCESS"
... }
>>>
>>>
>>>
>>> print(s3_file["bucket_name"])
company-raw-data
>>>
>>> print(s3_file["file_name"])
customers.csv
>>>
>>> print(s3_file["record_count"] + 500)
1500
>>> print(s3_file["status"])
SUCCESS
>>>

```

18. List of Dictionaries

A List of Dictionaries means:

- A List ([]) stores multiple items.
- Each item in the list is a Dictionary ({}).
- This is one of the most important data structures in Python, AWS, and Data Engineering.

Why is List of Dictionaries important in AWS?

AWS services such as:

- 1) S3
- 2) EC2
- 3) Lambda
- 4) DynamoDB
- 5) Boto3 APIs

often return data as JSON, which becomes a List of Dictionaries in Python.

```

>>>
>>> customers = [
...     {"id": 1, "name": "John", "city": "Chennai"},
...     {"id": 2, "name": "Jonathan", "city": "Bangalore"},
...     {"id": 3, "name": "Amy", "city": "Hydrabad"}
... ]
>>>
>>>
>>> for customer in customers:
...     print(customer["id"])
...
1
2
3
>>>     print(customer["name"])
File "<stdin>", line 1
    print(customer["name"])
IndentationError: unexpected indent
>>> for customer in customers:
...     print(customer["name"])
...
John
Jonathan
Amy
>>> for customer in customers:
...     print(customer["city"])
...
Chennai
Bangalore
Hydrabad

```

19.Process a List of Dictionaries

A List of Dictionaries becomes powerful when we process each dictionary using a loop.

This is a very common task in:

- AWS S3
- AWS Lambda
- DynamoDB
- Boto3
- ETL Pipelines
- API Responses

```
>>>
>>> customers = [
...     {"id": 1, "name": "John", "city": "Chennai"},
...     {"id": 2, "name": "Jonathan", "city": "Banglore"},
...     {"id": 3, "name": "Amy", "city": "Hydrabad"}
... ]
>>>
>>>
>>> for customer in customers:
...     print(customer["id"])
...
1
2
3
>>>     print(customer["name"])
      File "<stdin>", line 1
        print(customer["name"])
IndentationError: unexpected indent
>>> for customer in customers:
...     print(customer["name"])
...
John
Jonathan
Amy
>>> for customer in customers:
...     print(customer["city"])
...
Chennai
Banglore
Hydrabad
```

20.Real Data Engineering Scenario

Suppose our pipeline receives information about three files:

```
>>> files = [  
...     {  
...         "file_name": "customers.csv",  
...         "records": 1000,  
...         "status": "SUCCESS"  
...     },  
...     {  
...         "file_name": "orders.csv",  
...         "records": 2500,  
...         "status": "SUCCESS"  
...     },  
...     {  
...         "file_name": "products.csv",  
...         "records": 500,  
...         "status": "FAILED"  
...     }  
... ]  
>>>  
>>> for file in files:  
...     print(  
...         file["file_name"],  
...         file["records"],  
...         file["status"]  
...     )  
...  
customers.csv 1000 SUCCESS  
orders.csv 2500 SUCCESS  
products.csv 500 FAILED  
>>>  
>>>
```

Total records:

```
>>> total_records = 0
>>>
>>> for file in files:
...     total_records += file["records"]
...
>>> print(total_records)
4000
```

24. Practice 3 – Data Pipeline

A data pipeline is a sequence of steps that moves data from a source, processes it, and delivers it somewhere useful.

```
>>> pipeline = {
...     "pipeline_name": "customer_etl",
...     "source": "S3",
...     File "<stdin>", line 4
...     "source": "S3",
...     ^
SyntaxError: invalid syntax
>>> pipeline = {
...     "pipeline_name": "customer_etl",
...     "source": "S3",
...     "target": "Redshift",
...     "records": 5000,
...     "status": "SUCCESS"
... }
>>>
>>> print("Pipeline:", pipeline["pipeline_name"])
Pipeline: customer_etl
>>>
>>> print("Source:", pipeline["source"])
Source: S3
>>>
>>> print("Target:", pipeline["target"])
Target: Redshift
>>>
>>> print("Records:", pipeline["records"])
Records: 5000
>>>
>>> print("Status:", pipeline["status"])
Status: SUCCESS
>>>
```